# Round-Robin CPU Scheduling Using Circular Linked List

## 1. Title

Round-Robin CPU Scheduling Using Circular Linked List

## 2. Problem Statement

Develop a C++ program to implement the Round-Robin CPU Scheduling Algorithm using a Circular Linked List. The program should store process IDs and their burst times, accept a fixed time quantum, and execute processes in a circular order until all processes are completed. This program helps in understanding CPU scheduling and the practical use of Circular Linked Lists in operating systems.

## 3. Objectives

- To understand the concept of a Circular Linked List.
- To understand the working of the Round-Robin CPU Scheduling Algorithm.
- To create and manage process nodes dynamically.
- To store process IDs and their burst times.
- To accept a fixed time quantum from the user.
- To execute processes in a circular order.
- To repeat process execution until all processes are completed.

- To understand the application of linked lists in operating systems.

## 4. Algorithm

Step 1: Start the program.

Step 2: Create a Process node containing:

- Process ID
- Burst time
- Remaining burst time
- Pointer to the next process

Step 3: Initialize the Circular Linked List as empty.

Step 4: Enter the number of processes and their process IDs and burst times.

Step 5: Insert all processes into the Circular Linked List. Connect the last node to the first node.

Step 6: Enter the fixed time quantum.

Step 7: Select the first process and initialize the remaining burst time of each process.

Step 8: Execute the current process for the time quantum or until its remaining burst time becomes zero, whichever occurs first.

Step 9: Update the remaining burst time of the current process.

Step 10: If the process is completed, mark it as finished. Otherwise, keep it for its next turn.

Step 11: Move to the next process in the Circular Linked List.

Step 12: Repeat Steps 8–11 until all processes are completed.

Step 13: Display the process execution order.

Step 14: Stop the program.